

ML API

User Guide

Apache Spark MLlib · Clojure · REST

POST

/v1/ml/execute

Version 1.0 · Spark 3.5 · Clojure 1.11

Single unified endpoint for all Spark MLlib algorithm execution

Contents

1	Introduction	3
2	Quick Start	3
3	Algorithm Reference	4
3.1	ChiSquareTest	4
3.2	CountVectorizer	5
3.3	Tokenizer / RegexTokenizer	6
3.4	StopWordsRemover	7
3.5	NGram	8
3.6	StringIndexer	8
3.7	Binarizer	9
3.8	Normalizer	10
3.9	StandardScaler	10
3.10	Bucketizer	11
3.11	VectorAssembler	12
3.12	Imputer	12
3.13	ChiSqSelector	13
3.14	BucketedRandomProjectionLSH	14
3.15	GeneralizedLinearRegression	15
3.16	GeneralizedLinearRegressionModel	16
3.17	LDA	17
4	Error Handling	19

1 Introduction

The ML API is a Clojure-based REST service that exposes Apache Spark MLlib algorithms through a single, unified HTTP endpoint. Clients submit a JSON payload specifying the target algorithm and its parameters; the service executes the corresponding Spark job on the configured cluster and returns structured JSON results.

All 17 algorithms covered in this guide — spanning statistics, feature extraction, feature transformation, feature selection, locality-sensitive hashing, and full ML model training — are accessed through the same route:

POST	v1/ml/execute
-------------	---------------

The server connects to the Spark master at `spark://fc-bergin:7077`, uses `taoensso.timbre` for structured logging, reads configuration from `config/config.edn` via `omniconf`, and sets `spark.sql.shuffle.partitions` to 4 for the development cluster.

2 Quick Start

Generic Request Shape

```
{
  "algorithm": "<AlgorithmName>",
  "parameters": {
    "url": "/path/to/data.csv",
    // ... algorithm-specific parameters
  }
}
```

Generic Response Shape

```
{
  "status": "success",
  "algorithm": "<AlgorithmName>",
  "result": { ... }
}
```

Example — Imputer via cURL

```
curl -X POST /v1/ml/execute \
  -H "Content-Type: application/json" \
  -d '{
    "algorithm": "Imputer",
    "parameters": {
      "url": "/data/customer_data.csv",
      "feature_field": ["salary"],
      "output_field": ["imputed_salary"],
      "strategy": "mean",
      "missing_value": ["NaN"]
    }
  }'
```

Key Rules

- All CSV files must have a header row. Column types are inferred automatically by Spark.
- The "algorithm" field value is case-sensitive and must match the exact name listed in this guide.
- CSV paths in "url" must be accessible from the Spark driver and all worker nodes.
- Parameters marked Required = Yes will return an error if omitted.

3 Algorithm Reference

Algorithms are grouped by functional category. Each section includes a plain-language description, a ready-to-use JSON request body, and a full parameter table.

3.1 ChiSquareTest

Statistics

Performs Pearson's Chi-Square test of independence between every input feature and a categorical target label. Returns per-feature chi-square statistics, p-values, and degrees of freedom. Features with lower p-values are more predictive; those showing statistical independence with the label can safely be dropped.

Request Body

```
{
  "algorithm": "ChiSquareTest",
  "parameters": {
    "url": "/data/customer_data.csv",
    "feature_field": ["age", "monthly_income", "gender", "city", "product_category"],
    "label_field": "purchase_status",
    "flatten": true
  }
}
```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"ChiSquareTest"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/data.csv"
<code>feature_field</code>	Columns to evaluate (assembled internally into a feature vector)	List[String]	—	Yes	["age", "gender", "city"]
<code>label_field</code>	Target label column (must be categorical / numeric index)	String	—	Yes	"purchase_status"
<code>flatten</code>	true → one result row per feature; false → single vector result	Boolean	false	No	true

3.2 CountVectorizer

Feature Extractor

Converts tokenised text documents into sparse numerical count vectors. Scans the full corpus to build a vocabulary, then represents each document as a vector of term frequencies. Supports vocabulary-size limits, document-frequency filtering, and optional binary (presence/absence) output.

Request Body

```
{
  "algorithm": "CountVectorizer",
  "parameters": {
    "url": "/data/text_documents.csv",
    "input_field": "words",
    "output_field": "features",
    "word_limit": 5000,
    "min_docs": 2,
    "min_count": 1.0
  }
}
```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"CountVectorizer"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/docs.csv"
<code>input_field</code>	Column containing arrays of string tokens	String	—	Yes	"words"
<code>output_field</code>	Output column for the generated sparse feature vector	String	—	Yes	"features"
<code>word_limit</code>	Maximum vocabulary size to retain (min 1)	Integer	262144	No	5000
<code>min_docs</code>	Minimum number of documents a term must appear in to be kept (min 1)	Integer	1	No	2
<code>min_count</code>	Minimum term frequency per document for inclusion (min 0.0)	Float	1.0	No	1.0

3.3 Tokenizer / RegexTokenizer

Feature Transformer

Splits raw text into arrays of word tokens. Use algorithm: "Tokenizer" for simple whitespace splitting. Use algorithm: "RegexTokenizer" for custom regex-based splitting with lowercase conversion and minimum token length filtering. Both share the same endpoint.

Request Body

```
// Standard Tokenizer
{
  "algorithm": "Tokenizer",
  "parameters": {
    "url":          "/data/text_data.csv",
    "input_field":  "sentence",
    "output_field": "words"
  }
}

// Regex Tokenizer
{
  "algorithm": "RegexTokenizer",
  "parameters": {
    "url":          "/data/text_data.csv",
    "input_field":  "sentence",
    "output_field": "words",
    "pattern":      "\\W",
    "pattern_as_delimiter": true,
    "minimum_token_length": 2,
    "convert_to_lowercase": true
  }
}
```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	"Tokenizer" or "RegexTokenizer"	String	—	Yes	"RegexTokenizer"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/text.csv"
<code>input_field</code>	Column containing raw text sentences	String	—	Yes	"sentence"
<code>output_field</code>	Output column for the token arrays	String	—	Yes	"words"
<code>pattern</code>	Regex pattern (RegexTokenizer only)	String	"\s+"	No	"\W"
<code>pattern_as_delimiter</code>	true → pattern splits text; false → pattern extracts tokens	Boolean	true	No	true
<code>minimum_token_length</code>	Drop tokens shorter than this length (min 0; RegexTokenizer only)	Integer	1	No	2
<code>convert_to_lowercase</code>	Lowercase all text before tokenising (RegexTokenizer only)	Boolean	true	No	true

3.4 StopWordsRemover

Feature Transformer

Removes high-frequency, low-signal stop words (e.g. "a", "the", "is") from tokenised arrays. Uses Spark's built-in English stop-word list by default; pass `stop_words` to override with a custom list. Supports case-insensitive matching.

Request Body

```
{
  "algorithm": "StopWordsRemover",
  "parameters": {
    "url": "/data/text_data.csv",
    "input_field": "customer_review",
    "output_field": "filtered_review",
    "stop_words": ["a", "the", "is"],
    "case_sensitive": false
  }
}
```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"StopWordsRemover"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/text_data.csv"
<code>input_field</code>	Column containing tokenised word arrays	String	—	Yes	"words"
<code>output_field</code>	Output column for filtered token arrays	String	—	Yes	"filtered_words"
<code>stop_words</code>	Custom stop word list (overrides Spark defaults when given)	List[String]	Spark defaults	No	["a", "the", "is"]
<code>case_sensitive</code>	Enable case-sensitive matching	Boolean	false	No	false

3.5 NGram

Feature Transformer

Generates sequences of n consecutive words from a tokenised column. Each n -gram is returned as a space-separated string (e.g. $n=2$ produces bi-grams). Rows with fewer tokens than n yield empty arrays. Null tokens are silently ignored.

Request Body

```
{
  "algorithm": "NGram",
  "parameters": {
    "url": "/data/text_data.csv",
    "input_field": "review_text",
    "output_field": "generated_ngrams",
    "ngram_size": 2
  }
}
```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"NGram"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/text.csv"
<code>input_field</code>	Column containing tokenised word arrays	String	—	Yes	"words"
<code>output_field</code>	Output column for n-gram string arrays	String	—	Yes	"ngrams"
<code>ngram_size</code>	Number of consecutive words per gram (min 1)	Integer	2	No	2

3.6 StringIndexer

Feature Transformer

Encodes one or more categorical string columns into numeric index columns. By default, the most frequent category receives index 0. The ordering strategy and handling of previously unseen categories are both configurable.

Request Body

```
{
  "algorithm": "StringIndexer",
  "parameters": {
    "url": "/data/customer_data.csv",
    "input_field": ["city"],
    "output_field": ["city_index"],
    "invalid_value": "keep",
    "label_order": "frequencyDesc"
  }
}
```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"StringIndexer"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/data.csv"
<code>input_field</code>	Categorical column(s) to index	List[String]	—	Yes	["city"]
<code>output_field</code>	Output column(s) for numeric indexes (same order as input_field)	List[String]	—	Yes	["city_index"]
<code>invalid_value</code>	How to handle unseen categories at transform time: error skip keep	String	"error"	No	"keep"
<code>label_order</code>	Category ordering: frequencyDesc frequencyAsc alphabetAsc alphabetDesc	String	"frequencyDesc"	No	"frequencyDesc"

3.7 Binarizer

Feature Transformer

Converts continuous numeric values to binary (0.0 or 1.0) using a per-column threshold. Values strictly greater than the threshold become 1.0; all others become 0.0. Multiple columns can be binarized in one request with independent thresholds.

Request Body

```
{
  "algorithm": "Binarizer",
  "parameters": {
    "url": "/data/student_score.csv",
    "input_field": ["score"],
    "output_field": ["pass_student"],
    "threshold_values": [4.0]
  }
}
```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"Binarizer"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/scores.csv"
<code>input_field</code>	Numeric column(s) to binarize	List[String]	—	Yes	["score"]
<code>output_field</code>	Output column(s) for binary values	List[String]	—	Yes	["pass_student"]
<code>threshold_values</code>	Threshold per column ($-\infty$ to $+\infty$). Length must equal <code>input_field</code> length.	List[Float]	[0.0]	No	[4.0]

3.8 Normalizer

Feature Transformer

Rescales each feature vector to unit norm, preserving the relative proportions between feature values. The norm type is controlled by `norm_type`: 1 for L1 (Manhattan), 2 for L2 (Euclidean), or Inf for L-infinity norm. The feature columns are assembled into a vector internally.

Request Body

```
{
```

```

{
  "algorithm": "Normalizer",
  "parameters": {
    "url": "/data/customer_data.csv",
    "feature_field": ["age", "monthly_income", "purchase_count", "credit_score"],
    "output_field": "normalized_feature",
    "norm_type": 2
  }
}

```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"Normalizer"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/data.csv"
<code>feature_field</code>	Numeric columns to assemble and normalise	List[String]	—	Yes	["age", "income"]
<code>output_field</code>	Output column for the normalised feature vector	String	—	Yes	"normalised_features"
<code>norm_type</code>	Norm: 1 = L1, 2 = L2, Inf = L-infinity	Float	2.0	No	2

3.9 StandardScaler

Feature Transformer

Standardises feature vectors to zero mean and/or unit standard deviation. Prevents large-magnitude features from dominating smaller ones during model training. `with_std` and `with_mean` can be combined independently.

Request Body

```

{
  "algorithm": "StandardScaler",
  "parameters": {
    "url": "/data/customer_data.csv",
    "feature_field": ["age", "monthly_income", "purchase_count", "credit_score"],
    "output_field": "scaled_features",
    "with_std": true,
    "with_mean": false
  }
}

```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"StandardScaler"

Parameter	Description	Type	Default	Required	Example
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/data.csv"
<code>feature_field</code>	Numeric columns to scale	List[String]	—	Yes	["age", "income"]
<code>output_field</code>	Output column for the scaled feature vector	String	—	Yes	"scaled_features"
<code>with_std</code>	Divide each feature by its standard deviation	Boolean	true	No	true
<code>with_mean</code>	Subtract the mean before scaling (centres to zero)	Boolean	false	No	false

3.10 Bucketizer

Feature Transformer

Maps continuous numeric values to discrete bucket indexes using user-defined split boundaries. Each set of splits must begin with a value representing negative infinity and end with a value representing positive infinity. Multiple columns can be bucketed independently in one call.

Request Body

```
{
  "algorithm": "Bucketizer",
  "parameters": {
    "url": "/data/customer_data.csv",
    "feature_field": ["salary"],
    "output_field": ["salary_bucket"],
    "bucket_splits": [[-999999, 30000, 80000, 999999]],
    "invalid_value": "keep"
  }
}
```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"Bucketizer"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/data.csv"
<code>feature_field</code>	Numeric column(s) to bucket	List[String]	—	Yes	["salary"]
<code>output_field</code>	Output column(s) for bucket indexes	List[String]	—	Yes	["salary_bucket"]
<code>bucket_splits</code>	Boundary values per column. Must span -Inf to +Inf.	List[List[Float]]	—	Yes	[[[-999999, 30000, 80000, 999999]]]

Parameter	Description	Type	Default	Required	Example
<code>invalid_value</code>	Handle NaN/invalid values: error skip keep	String	"error"	No	"keep"

3.11 VectorAssembler

Feature Transformer

Concatenates multiple numeric, boolean, or vector columns into a single feature vector column. The order of values in the output vector mirrors the order of `input_field` entries. This is typically the final preprocessing step before passing data to an MLlib estimator.

Request Body

```
{
  "algorithm": "VectorAssembler",
  "parameters": {
    "url": "/data/customer_data.csv",
    "feature_field": ["age", "monthly_income", "purchase_score"],
    "output_field": "features",
    "invalid_value": "keep"
  }
}
```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"VectorAssembler"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/data.csv"
<code>feature_field</code>	Input columns to assemble into a vector	List[String]	—	Yes	["age", "income", "score"]
<code>output_field</code>	Output column for the assembled vector	String	—	Yes	"features"
<code>invalid_value</code>	Handle invalid/NaN values: error skip keep	String	"error"	No	"keep"

3.12 Imputer

Feature Transformer

Replaces missing numeric values using a statistical strategy computed column-by-column from non-missing records. Supports mean, median, and mode strategies. Only numeric columns are supported — categorical columns must be indexed first.

Request Body

```
{
  "algorithm": "Imputer",
  "parameters": {
    "url": "/data/customer_data.csv",
    "feature_field": ["salary"],
    "output_field": ["imputed_salary"],
    "strategy": "mean",
    "missing_value": ["NaN"]
  }
}
```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"Imputer"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/data.csv"
<code>feature_field</code>	Numeric column(s) to impute	List[String]	—	Yes	["salary"]
<code>output_field</code>	Output column(s) for imputed values	List[String]	—	Yes	["imputed_salary"]
<code>strategy</code>	Replacement strategy: mean median mode	String	"mean"	No	"median"
<code>missing_value</code>	Values treated as missing (e.g. NaN, null, 0, -1)	List[String/Float]	NaN	No	["NaN", "null", 0, -1]
<code>relative_error</code>	Relative approximation error for median (0.0 – 1.0)	Float	0.0001	No	0.0001

3.13 ChiSqSelector

Feature Selector

Selects the most statistically significant features using the Chi-Square test of independence against the target label. Supports five selection strategies: by top-N count, by percentile, or by false-positive / false-discovery / family-wise error rate thresholds.

Request Body

```
{
  "algorithm": "ChiSqSelector",
  "parameters": {
    "url": "/data/customer_data.csv",
    "feature_field": ["age", "salary", "purchase_score", "transaction_count"],
    "target_field": "purchase_status",
    "output_field": "selected_features",
    "selection_method": "numTopFeatures",
    "top_feature": 2
  }
}
```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"ChiSqSelector"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/data.csv"
<code>feature_field</code>	Feature columns to evaluate	List[String]	—	Yes	["age", "salary"]
<code>target_field</code>	Target label column (must be numeric index)	String	"label"	No	"purchase_status"
<code>output_field</code>	Output column for the selected feature vector	String	—	Yes	"selected_features"
<code>selection_method</code>	numTopFeatures percentile fpr fdr fwe	String	"numTopFeatures"	No	"percentile"
<code>top_feature</code>	Number of top features to keep (numTopFeatures mode; min 1)	Integer	50	No	2
<code>selection_percentage</code>	Fraction of features to keep (percentile mode; 0.0 – 1.0)	Float	0.1	No	0.2
<code>fpr_threshold</code>	False positive rate threshold (fpr mode; 0.0 – 1.0)	Float	0.05	No	0.02
<code>fdr_threshold</code>	False discovery rate threshold (fdr mode; 0.0 – 1.0)	Float	0.05	No	0.06
<code>fwe_threshold</code>	Family-wise error rate threshold (fwe mode; 0.0 – 1.0)	Float	0.05	No	0.04

3.14 BucketedRandomProjectionLSH

LSH Algorithm

Locality-sensitive hashing for approximate nearest-neighbour search using Euclidean distance. Projects feature vectors into hash buckets so geometrically similar vectors land together with high probability. Useful for similarity joins and fast retrieval without exhaustive distance calculations.

Request Body

```
{
  "algorithm": "BucketedRandomProjectionLSH",
  "parameters": {
    "url": "/data/product.csv",
    "feature_field": ["age", "job_score", "salary"],
    "output_field": "hashes",
    "bucket_length": 2.0,
    "num_hash_tables": 3,
    "random_seed": 12345
  }
}
```

}

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"BucketedRandomProjectionLSH"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/product.csv"
<code>feature_field</code>	Input feature columns	List[String]	—	Yes	["age", "job_score", "salary"]
<code>output_field</code>	Output column for hash bucket arrays	String	—	Yes	"hashes"
<code>bucket_length</code>	Width of each projection bucket (must be > 0.0)	Float	—	Yes	2.0
<code>num_hash_tables</code>	Number of hash tables — higher improves recall (min 1)	Integer	1	No	3
<code>random_seed</code>	Seed for reproducible random projections (1 – 2 ⁶³ –1)	Integer	—	No	12345

3.15 GeneralizedLinearRegression

ML Algorithm — Training

Trains a Generalized Linear Model (GLM) supporting Gaussian, Binomial, Poisson, Gamma, and Tweedie distribution families with configurable link functions. The fitted model is serialised to disk for reuse via `GeneralizedLinearRegressionModel`.

Request Body

```
{
  "algorithm": "GeneralizedLinearRegression",
  "parameters": {
    "url": "/data/glr_data.csv",
    "feature_field": ["age", "salary", "purchase_score"],
    "target_field": "purchase_amount",
    "output_field": "predicted_purchase",
    "model_path": "/tmp/glr_model",
    "model_label": "customer_purchase_glr_v1",
    "train_size": 80,
    "seed": 123,
    "regression_family": "gaussian",
    "prediction_link": "identity",
    "include_intercept": true,
    "max_iterations": 25,
    "training_tolerance": 0.000001,
    "regularization_strength": 0.0,
    "training_solver": "irls",
    "aggregation_depth": 2
  }
}
```

```
}
}
```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"Generalized Linear Regression"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/data.csv"
<code>feature_field</code>	Feature columns for training	List[String]	—	Yes	["age", "salary"]
<code>target_field</code>	Target label column	String	"label"	Yes	"purchase_amount"
<code>output_field</code>	Output column for predicted values	String	"prediction"	No	"predicted_purchase"
<code>model_path</code>	Directory to save the trained model	String	—	No	"/tmp/glr_model"
<code>model_label</code>	Human-readable model name	String	—	No	"glr_v1"
<code>train_size</code>	Percentage of data used for training (1–100)	Integer	80	No	80
<code>seed</code>	Random seed for reproducible data splits	Integer	123	No	123
<code>regression_family</code>	Distribution family: gaussian binomial poisson gamma tweedie	String	"gaussian"	No	"gaussian"
<code>prediction_link</code>	Link function: identity log inverse sqrt	String	Family default	No	"identity"
<code>include_intercept</code>	Include an intercept term in the model	Boolean	true	No	true
<code>max_iterations</code>	Maximum training iterations (min 1)	Integer	25	No	25
<code>training_tolerance</code>	Convergence threshold — training stops when improvement < this (min 0.0)	Float	0.000001	No	0.000001
<code>regularization_strength</code>	L2 regularisation strength (min 0.0)	Float	0.0	No	0.0
<code>training_solver</code>	Optimisation solver (currently only irls supported)	String	"irls"	No	"irls"
<code>aggregation_depth</code>	Tree depth for distributed aggregation (min 2)	Integer	2	No	2
<code>link_output_field</code>	Optional column for link-function predicted values	String	—	No	"link_prediction"
<code>offset_field</code>	Column with known offsets added to the linear predictor	String	—	No	"offset_value"
<code>row_weight_field</code>	Column with per-row training weights	String	—	No	"sample_weight"

3.16 GeneralizedLinearRegressionModel

ML Algorithm — Prediction

Loads a previously saved GeneralizedLinearRegression model from disk and generates predictions on a new dataset — no retraining required. The loaded model retains all coefficients, intercept, and statistical parameters from the original training run.

Request Body

```
{
  "algorithm": "GeneralizedLinearRegressionModel",
  "parameters": {
    "model_path": "/tmp/glr_model",
    "url": "/data/new_customer_data.csv",
    "feature_field": ["age", "salary", "purchase_score"],
    "target_field": "purchase_amount",
    "output_field": "predicted_purchase"
  }
}
```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"Generalized LinearRegressionModel"
<code>url</code>	Local path of the input CSV dataset for prediction	String	—	Yes	"/data/new_data.csv"
<code>model_path</code>	Path to the saved trained model directory	String	—	Yes	"/tmp/glr_model"
<code>feature_field</code>	Feature columns — must match training configuration	List[String]	—	Yes	["age", "salary", "purchase_score"]
<code>target_field</code>	Optional actual label column (used for evaluation metrics)	String	"label"	No	"purchase_amount"
<code>output_field</code>	Output column for predicted values	String	"prediction"	No	"predicted_purchase"

3.17 LDA

ML Algorithm — Topic Modelling

Latent Dirichlet Allocation discovers latent topics within a text corpus. The service handles the full preprocessing pipeline internally (Tokenizer → StopWordsRemover → CountVectorizer) before running LDA. Returns per-document topic probability distributions.

Request Body

```
{
  "algorithm": "LDA",
  "parameters": {
    "url": "/data/customer_reviews.csv",
    "text_field": "review_text",
    "topic_count": 10,
    "max_iterations": 20,
    "random_seed": 12345,
    "training_method": "online",
    "checkpoint_interval": 10,
    "initial_learning_offset": 1024,
    "learning_decay_rate": 0.51,
    "training_sample_rate": 0.05,
    "auto_optimize": true,
    "topic_distribution_field": "topic_distribution",
    "keep_last_checkpoint": true
  }
}
```

Parameters

Parameter	Description	Type	Default	Required	Example
<code>algorithm</code>	Name of the algorithm	String	—	Yes	"LDA"
<code>url</code>	Local path of the input CSV dataset	String	—	Yes	"/data/reviews.csv"
<code>text_field</code>	Column containing raw document text	String	—	Yes	"review_text"
<code>topic_count</code>	Number of topics to discover (min 1)	Integer	10	No	10
<code>max_iterations</code>	Maximum training iterations (min 1)	Integer	20	No	20
<code>random_seed</code>	Seed for reproducible topic initialisation	Integer	—	No	12345
<code>training_method</code>	online (mini-batch) em (full expectation-maximisation)	String	"online"	No	"online"
<code>checkpoint_interval</code>	Iterations between training checkpoints (min 1)	Integer	10	No	10
<code>initial_learning_offset</code>	Controls early-iteration step size (> 0.0)	Float	1024.0	No	1024.0
<code>learning_decay_rate</code>	Learning rate decay for online optimisation (0.5 – 1.0]	Float	0.51	No	0.51
<code>training_sample_rate</code>	Fraction of corpus sampled per iteration (0.0 – 1.0)	Float	0.05	No	0.05
<code>auto_optimize</code>	Automatically optimise document-topic concentration	Boolean	true	No	true
<code>document_concentration</code>	Dirichlet prior for document-topic distribution (> 0)	List[Float]	—	No	[0.1]
<code>topic_concentration</code>	Dirichlet prior for topic-word distribution (> 0.0)	Float	—	No	0.1
<code>topic_distribution_field</code>	Output column for per-document topic probability vectors	String	"topicDistribution"	No	"topic_distribution"
<code>keep_last_checkpoint</code>	Retain the final training checkpoint on disk	Boolean	true	No	true

Parameter	Description	Type	Default	Required	Example
kpoint					

4 Error Handling

When a request fails the service returns HTTP 4xx/5xx with a JSON error body:

```
{
  "status": "error",
  "algorithm": "<AlgorithmName>",
  "error": "Human-readable description of what went wrong"
}
```

Common Error Causes

Parameter	Description	Type	Default	Required	Example
Unknown algorithm	The "algorithm" field does not match any registered handler	—	—	—	—
Missing required field	A parameter marked Required = Yes is absent	—	—	—	—
File not found	The path in "url" is not accessible from the Spark cluster nodes	—	—	—	—
Type mismatch	A parameter value has the wrong JSON type (e.g. string instead of integer)	—	—	—	—
Column not found	A column named in feature_field, label_field, etc. does not exist in the CSV	—	—	—	—
Spark execution error	The Spark job itself failed — check server logs for a Java stack trace	—	—	—	—
Model path missing	model_path not found or empty when calling the Prediction variant	—	—	—	—